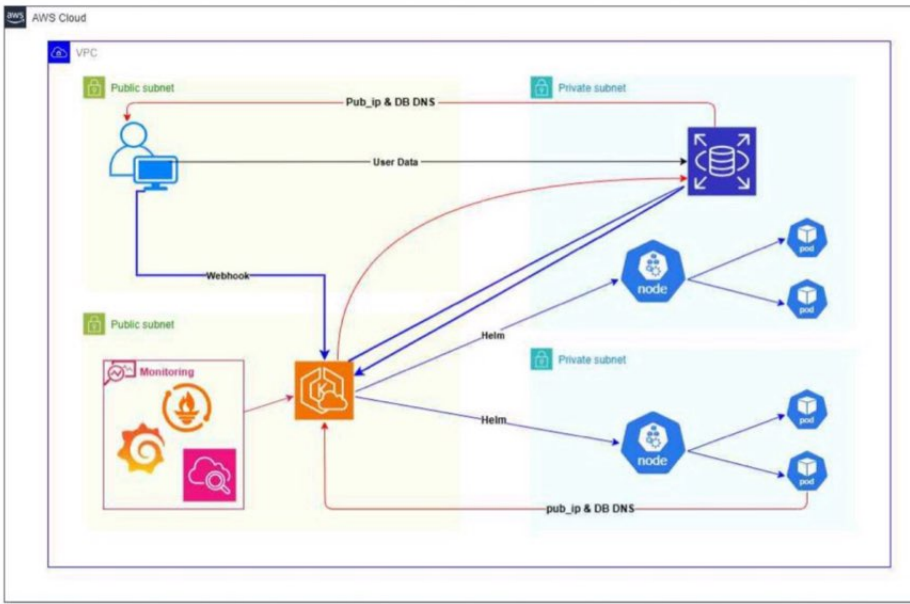
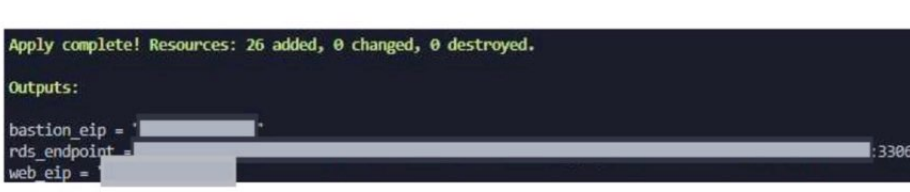


프로젝트 요약서 1

개요			
프로젝트명	AWS 환경에서 Helm & Shell 스크립트를 활용한 Kubernetes 자동화 배포 시스템 구축	구분	팀
프로젝트 목적	운영 비용 절감 & 배포 시간 단축 사용자별 배포 자동화 모니터링 및 문제 대응 체계 강화		
프로젝트 기대효과	Terraform을 이용한 운영 비용 절감 & 배포 시간 단축 Shell Script & Helm을 이용하여 사용자별 배포 자동화 서비스 신청 페이지에서 Pod의 리소스를 선택하게 하여 낭비 최소화 및 최적화 Grafana & Cloud Watch로 모니터링 및 문제 대응 체계 강화		
담당업무(팀프로젝트시 작성)	팀장 Terraform, Shell Script, Helm, eks	기간	24.12.11 ~ 25.01.09

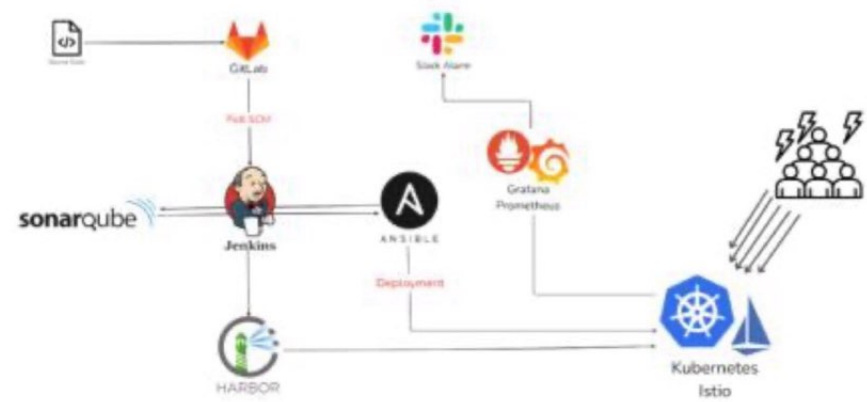
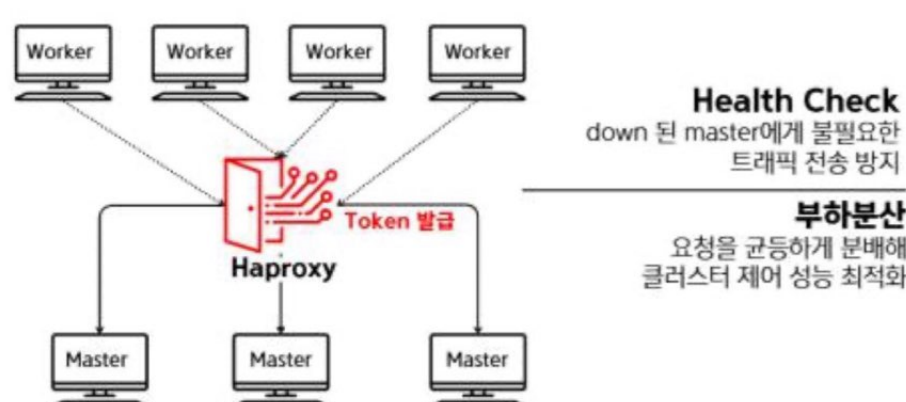
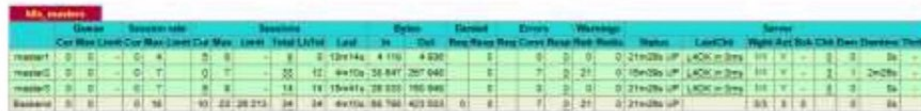
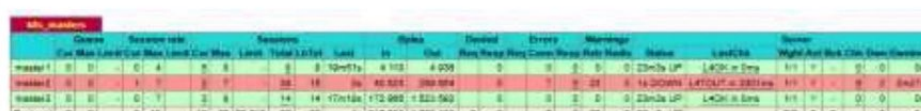
세부내용

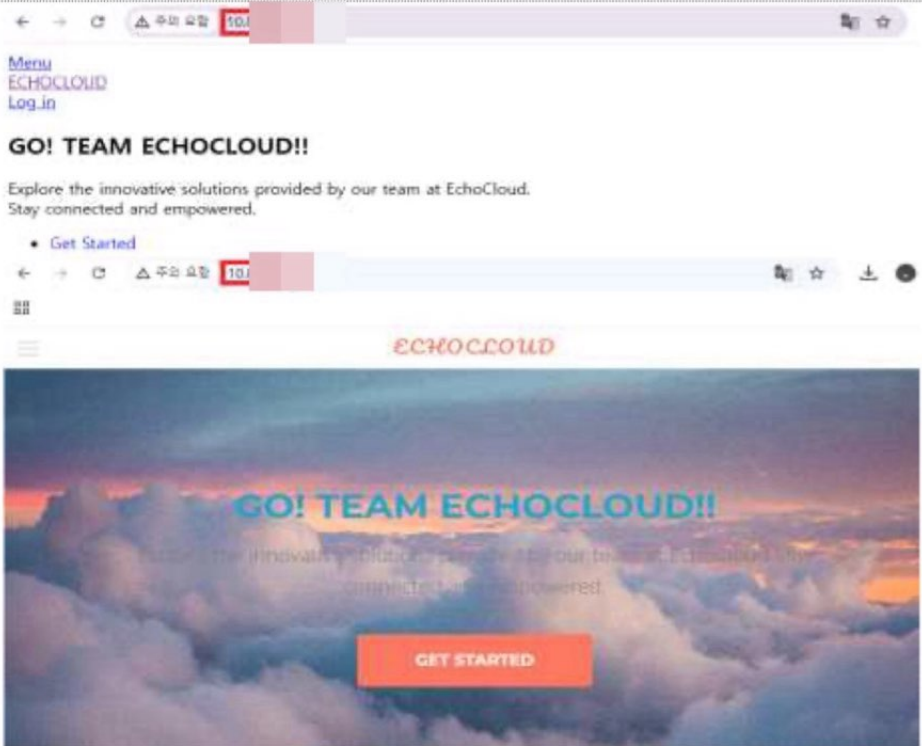
구성 및 작성 내용 설명 1	Terraform으로 모든 AWS Resource 생성 WebSrv -> RDS로 user_data 전송 & Bastion으로 Webhook 실행 Webhook을 받은 Bastion은 RDS로부터 Data 값을 변수로 담아 사용자별로 Helm Rep 생성 배포된 서비스들의 external_ip, db.dns 주소를 RDS로 전송 & WebSrv에서 해당 output 확인 Grafana & Cloud Watch로 eks-cluster, namespace, pod, svc 등 각종 서비스 모니터링
자료(이미지 외)	
구성 및 작성 내용 설명 2	Terraform을 사용하여 aws 리소스 생성(ec2-instance, RDS-instance) & output 확인
자료(이미지 외)	 <pre> Apply complete! Resources: 26 added, 0 changed, 0 destroyed. Outputs: bastion_eip = [REDACTED] rds_endpoint = [REDACTED] web_eip = [REDACTED]:3306" </pre>

구성 및 작성 내용 설명 3 자료(이미지 외)	Node.js로 RDS와 연결하여 웹 서비스 신청 페이지 구현 & output으로 접속 주소 확인 <pre> root@[redacted]:/mywebhook# ls db.js deployment.tog node_modules package-lock.json package.json webhook.js root@[redacted]:/mywebhook# node webhook.js ☑ Webhook listener is running ☑ MySQL 원격 연결 성공 root@[redacted]:/nodejs_Pro# ls app.js css img node_modules package-lock.json package.json testwebhook.js views root@[redacted]:/nodejs_Pro# node app.js [redacted] 서버가 실행 중입니다 MySQL 원격 연결 성공 </pre> 구성 및 작성 내용 설명 4 Shell 파일의 세분화 (역할 세분화) 자료(이미지 외) <pre> root@Master1:~/lab# cat main.sh #!/bin/bash # 변수 가져오기 source ./fetch_users.sh # 사용자 처리 source ./process_user.sh # 헬름 생성 source ./helm_setup.sh # hash 값 비교 source ./deployment_check.sh # external_ip & db.dns 주소 RDS로 Insert source ./external_ip.sh </pre> 구성 및 작성 내용 설명 5 Shell파일 실행 (RDS 데이터 가져오기 -> 사용자별 Helm 배포 -> output RDS로 전송) 자료(이미지 외) <pre> ☑ Token Validated, Executing Script... ☑ Script STDOUT: 데이터베이스에서 사용자 정보를 가져왔습니다: user1 wp v1 1.00 1024 ☑ 사용자 정보 확인 중... user1 wp v1 1.00 1024 ☑ 사용자 [user1]: 애플리케이션 [wp] (버전: v1) CPU: 1.00 Core RAM: 1024MB 처리 완료. creating user1 ☑ [user1]의 Helm 차트를 새로 생성했습니다. ! [user1]의 불필요한 템플릿 파일을 삭제합니다. ☑ [user1]의 불필요한 파일을 정리했습니다. ☑ [user1]의 템플릿 디렉터리에 deployment.yaml 및 service.yaml을 복사합니다. ☑ [user1]에 사용자 정의 템플릿 파일을 복사했습니다. ☑ [user1]의 values.yaml 파일을 설정합니다. ☑ [user1]의 values.yaml 파일이 성공적으로 업데이트되었습니다. ☑ [user1]에 변경사항이 감지되었습니다. Helm 배포를 진행합니다. Release "user1-helm" does not exist. Installing it now. NAME: user1-helm LAST DEPLOYED: Mon Jan 6 06:58:08 2025 NAMESPACE: web-user1 STATUS: deployed REVISION: 1 ☑ [user1]의 Helm 배포가 완료되었습니다. ☑ [user1] External IP 가져오기 (시도: 1/5)... ! External IP가 아직 할당되지 않았습니다. 10초 후 재시도... ☑ [user1] External IP 가져오기 (시도: 2/5) ☑ External IP: [redacted] ☑ [redacted] 할당 접속 주소 [redacted] ☑ [user1]의 External IP를 데이터베이스에 저장합니다... ☑ [user1]의 External IP가 데이터베이스에 성공적으로 저장되었습니다. ☑ [user1]의 DB 도메인 주소를 데이터베이스에 저장합니다... ☑ [user1]의 DB 도메인 주소가 데이터베이스에 성공적으로 저장되었습니다. </pre>
---	--

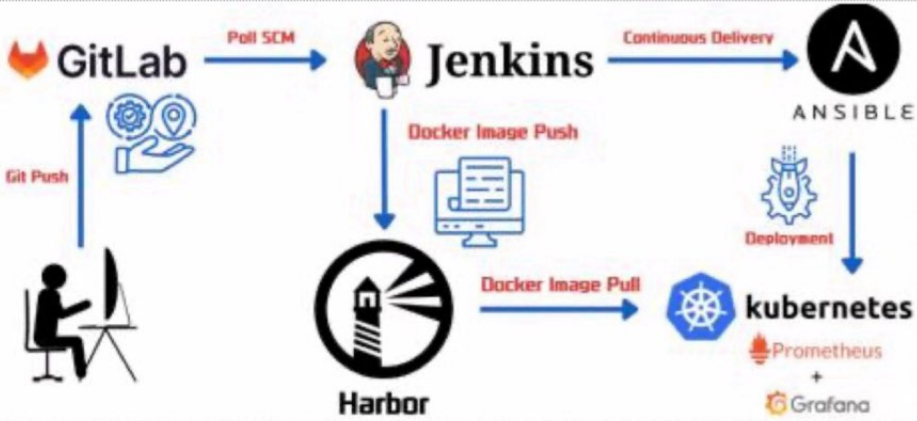
구성 및 작성 내용 설명 6 자료(이미지 외)	Helm의 values.yaml을 템플릿화 (sed -i 명령어와 함께 사용) Shell Script helm_setup 과 values.yaml로 사용자별로 배포 <pre> username: USER_ID containerName: APP image: repository: doro0704/APP pullPolicy: IfNotPresent tag: VERSION service: type: LoadBalancer port: 80 db: enabled: true image: mysql:5.7 containerName: db service: type: ClusterIP port: 3306 resources: cpu: CPU ram: RAM </pre> <pre> # deployment.yaml 및 service.yaml 복사 21 echo "[\${user_id}]의 템플릿 디렉토리에 deployment.yaml 및 service.yaml 복사합니다." 22 cp deployment.yaml "\${user_id}/templates/deployment.yaml" 23 cp service.yaml "\${user_id}/templates/service.yaml" 24 echo "[\${user_id}]에 사용자 정의 템플릿 파일을 복사했습니다." # values.yaml 템플릿 파일 설정 25 echo "[\${user_id}]의 values.yaml 파일을 설정합니다." 26 cp values.yaml "\${user_id}/values.yaml" 27 sed -i "s APP \${app} g" "\${user_id}/values.yaml" 28 sed -i "s VERSION \${ver} g" "\${user_id}/values.yaml" 29 sed -i "s USER_ID \${user_id} g" "\${user_id}/values.yaml" 30 sed -i "s CPU \${cpu} g" "\${user_id}/values.yaml" 31 sed -i "s RAM \${ram} g" "\${user_id}/values.yaml" 32 echo "[\${user_id}]의 values.yaml 파일이 성공적으로 업데이트되었습니다." </pre>										
구성 및 작성 내용 설명 7 자료(이미지 외)	신청한 서비스가 WordPress/XpressEngine + DB pod가 올라가는 것을 확인 namespace도 별도로 생성되며, pod 이름도 사용자 명 기반으로 생성 <pre> root@~: /lab# k get ns NAME STATUS AGE amazon-cloudwatch Active 94m default Active 124m kube-node-lease Active 124m kube-public Active 124m kube-system Active 124m monitoring Active 80m web-user1 Active 12m web-user2 Active 47s root@~: /lab# k get pod,svc -n web-user2 NAME READY STATUS RESTARTS AGE pod/user2-xx-8k8gs 1/1 Running 0 61s pod/user2-xe-7c80050c06-t98f6 1/1 Running 0 61s NAME TYPE CLUSTER-IP EXTERNAL-IP service/user2-db-svc ClusterIP 10.0.0.1 <none> service/user2-svc LoadBalancer 10.0.0.2 <none> </pre>										
구성 및 작성 내용 설명 8 자료(이미지 외)	eks-cluster & 신청한 서비스 및 리소스를 Grafana와 Cloud Watch로 확인 <table border="1"> <thead> <tr> <th>Time</th> <th>(namespace="web-legend")</th> </tr> </thead> <tbody> <tr> <td>2025-09-06 05:22:30</td> <td>2</td> </tr> <tr> <td>2025-09-06 05:23:00</td> <td>2</td> </tr> <tr> <td>2025-09-06 05:23:30</td> <td>2</td> </tr> <tr> <td>2025-09-06 05:24:00</td> <td>2</td> </tr> </tbody> </table>	Time	(namespace="web-legend")	2025-09-06 05:22:30	2	2025-09-06 05:23:00	2	2025-09-06 05:23:30	2	2025-09-06 05:24:00	2
Time	(namespace="web-legend")										
2025-09-06 05:22:30	2										
2025-09-06 05:23:00	2										
2025-09-06 05:23:30	2										
2025-09-06 05:24:00	2										

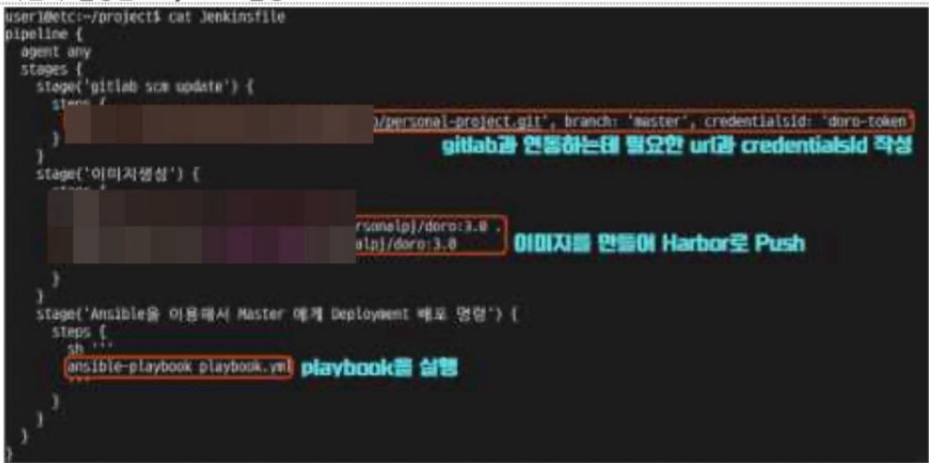
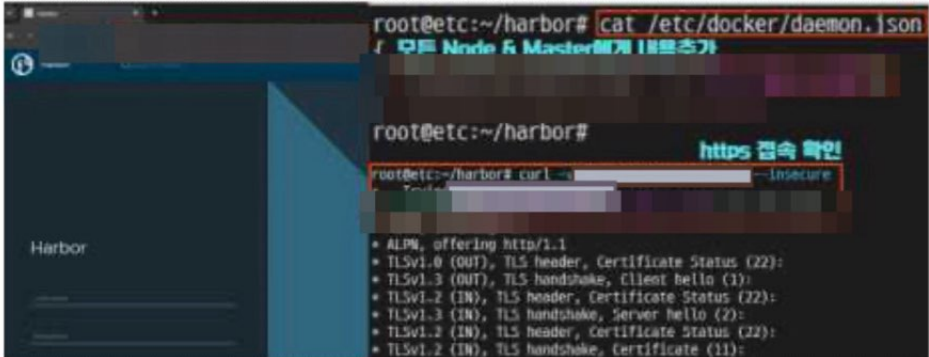
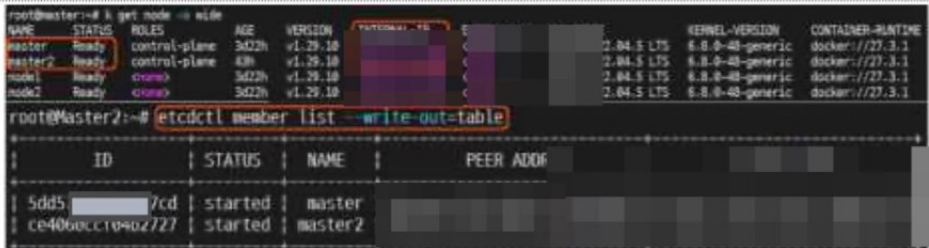
프로젝트 요약서 2

개요			
프로젝트명	Istio 기반의 Canary Deployment와 CI/CD 파이프라인 구현	구분	팀
프로젝트 목적	CI/CD 자동화를 통해 효율적인 배포와 관리체계 확립 Istio의 트래픽 라우팅으로 무중단배포 구현, 배포 신뢰성 향상 문제 해결 자동화 및 실시간 Slack 알림 연동		
프로젝트 기대효과	Harbor(https) 와 Istio 로 보안 강화 CI/CD 자동화 및 SonarQube를 사용하여 운영의 효율성 증가 Canary Update로 배포의 신뢰성 향상 (무중단)		
담당업무(팀프로젝트시 작성)	팀장 Istio, Multi-Master, CI/CD 구축	기간	24.11.25 ~ 24.12.10
세부내용			
구성 및 작성 내용 설명 1	개발자가 Git으로 Push하고 Jenkins는 PollSCM으로 매일 오후 5시45분에 자동으로 Build SonarQube는 빌드전에 Jenkins가 가져온 파일을 검사하고, Ansible은 Manifest 파일을 복사하여 배포 배포된 Deployment 에게 Stress를 주고 Slack으로 알람이 오게끔 Grafana와 연동 		
자료(이미지 외)			
구성 및 작성 내용 설명 2	LoadBalancer을 이용한 Multi-Master 구조를 만들었고, 토큰 발급시 LB 주소로 발급 LB는 Master에게 계속 Health Check를 하며, 하나의 Master에게 요청하지 않도록 부하분산  부하분산 요청을 균등하게 분배해 클러스터 제어 성능 최적화		
자료(이미지 외)	 		

구성 및 작성 내용 설명 3	Jenkins에서 자동으로 Build하여 Ansible-Playbook을 통해 Pod를 자동으로 배포
자료(이미지 외)	<pre> * ansible-playbook master.yml PLAY [선택 파일 복사 및 kubectl 실행] ***** TASK [Gathering Facts] ***** ok: [] TASK [파일 복사] ***** changed: [] TASK [파일 복사] ***** changed: [] TASK [파일 복사] ***** changed: [] TASK [kubectl 명령 실행] ***** changed: [] PLAY RECAP ***** : ok=6 changed=5 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0 </pre>
구성 및 작성 내용 설명 4	같은 IP를 접속해도, 서로 다른 페이지가 뜨게끔 설정 (특정 ip만 v2 페이지 라우팅) Istio에서 VirtualService와 DestinationRule 설정
자료(이미지 외)	
구성 및 작성 내용 설명 5	K6를 이용하여 일부러 Stress를 주고, hpa 설정으로 Pod를 늘리게 설정 Slack & Grafana 연동으로 Pod의 개수가 증가하면 알림이오고, 줄어들면 다시 알림이 오게 설정
자료(이미지 외)	

프로젝트 요약서 3

개요			
프로젝트명	CI/CD 파이프라인 자동화	구분	개인
프로젝트 목적	CI/CD 자동화를 통해 효율적인 배포와 관리체계 확립 사설저장소를 이용하고 보안성강화 및 효율적인 관리 Master을 하나 더 추가하여 HA 확보		
프로젝트 기대효과	CI/CD 자동화 및 SonarQube를 사용하여 운영의 효율성 증가 Harbor를 https로 구성하여 보안성 증가 및 효율적인 이미지 관리 Multi-Master로 HA확보 및 etcd backup으로 복구 능력 향상		
담당업무(팀프로젝트시 작성)		기간	24.11.4 ~ 24.11.11
세부내용			
구성 및 작성 내용 설명 1	개발자가 Git으로 Push하고 Jenkins는 PollSCM으로 20분마다 자동으로 Build Jenkins가 가져온 파일을 검사하고, Ansible은 Manifest 파일을 복사하여 배포 Prometheus & Grafana를 이용하여 실시간 CPU 사용량 확인 및 모니터링		
자료(이미지 외)	 <pre> graph LR User((User)) -- "Git Push" --> GitLab[GitLab] GitLab -- "Poll SCM" --> Jenkins[Jenkins] Jenkins -- "Continuous Delivery" --> Ansible[ANSIBLE] Jenkins -- "Docker Image Push" --> Harbor[Harbor] Harbor -- "Docker Image Pull" --> Kubernetes[kubernetes] Ansible -- "Deployment" --> Kubernetes subgraph Monitoring Prometheus[Prometheus] Grafana[Grafana] end Kubernetes --- Monitoring </pre>		
구성 및 작성 내용 설명 2	/etc/ansible/hosts에 master을 등록 Ansible Playbook.yml 파일로 Jenkins에게 있는 manifest 파일을 복사하여 배포하게 설정 Jenkins Console 창에서 공인IP를 확인 할 수 있게 output 설정		
자료(이미지 외)	<pre> user1@etc:~/project\$ cat playbook.yml --- - name: Playbook 과 Jenkins를 이용한 CI/CD hosts: master become: true tasks: - name: Master에게 yml file 복사 copy: src: deployment.yml dest: /home/user1/project_jenkins/deployment.yml owner: user1 group: user1 mode: '0664' - name: deployment 배포 shell: sudo kubectll apply -f /home/user1/project_jenkins/deployment.yml - name: Services 목록 출력 shell: sudo kubectll get svc register: svc_output - name: 배포한 Deployment 공인 IP 확인 </pre> Ansible Playbook.yml 파일로 Jenkins에게 있는 manifest 파일을 복사하여 배포하게 설정 Jenkins Console 창에서 공인IP를 확인 할 수 있게 output 설정		

구성 및 작성 내용 설명 3	Jenkinsfile에서 gitlab과 연동하기 위한 credentialsID 작성 Harbor에게 Image Build후 Push 이전에 설명한 Playbook 실행																																																									
자료(이미지 외)	 user@etc:~/project\$ cat Jenkinsfile <pre> pipeline { agent any stages { stage('gitlab scm update') { steps { //personal-project.git', branch: 'master', credentialsId: 'doro-token } } stage('이미지생성') { steps { //doro:3.0 //alp/doro:3.0 } } stage('Ansible을 이용해서 Master 에게 Deployment 배포 명령') { steps { sh ''' ansible-playbook playbook.yml ''' } } }</pre> gitlab과 연동하는데 필요한 uri과 credentialsid 작성 이미지를 만들어 Harbor로 Push playbook을 실행																																																									
구성 및 작성 내용 설명 4	Harbor를 사용하여 컨테이너 이미지를 효율적으로 관리하고 접근을 제어 HTTPS 인증서를 설정하여 전송 데이터의 암호화 & 외부 공격에 대한 보안 강화 역할 기반 접근 제어(RBAC)로 사용자 권한을 세밀하게 관리																																																									
자료(이미지 외)	 root@etc:~/harbor# cat /etc/docker/daemon.json <pre> { //모든 Node & Master에게 배포 }</pre> https 접속 확인 root@etc:~/harbor# curl -s -k https://192.168.1.10:443 ALPN, offering http/1.1 • TLSv1.0 (OUT), TLS header, Certificate Status (22): • TLSv1.3 (OUT), TLS handshake, Client hello (1): • TLSv1.2 (IN), TLS header, Certificate Status (22): • TLSv1.3 (IN), TLS handshake, Server hello (2): • TLSv1.2 (IN), TLS header, Certificate Status (22): • TLSv1.2 (IN), TLS handshake, Certificate (11):																																																									
구성 및 작성 내용 설명 5	기존에 있는 k8s cluster에게서 새로운 token을 발급하고 Multi-Master 구성 리소스 한계로 인해 k8s 멀티 마스터 환경을 LoadBalancer 없이 구성하여 두 개의 마스터 노드를 클러스터로 묶어 HA를 확보																																																									
자료(이미지 외)	 root@master1:~# k get node -o wide <table><tr><th>NAME</th><th>STATUS</th><th>ROLES</th><th>AGE</th><th>VERSION</th><th>IP</th><th>OS</th><th>CONTAINER-ENGINE</th><th>CONTAINER-RUNTIME</th></tr><tr><td>master</td><td>Ready</td><td>control-plane</td><td>3d22h</td><td>v1.29.10</td><td>192.168.1.10</td><td>Ubuntu 22.04.5 LTS</td><td>6.8.0-40-generic</td><td>docker://27.3.1</td></tr><tr><td>master2</td><td>Ready</td><td>control-plane</td><td>43h</td><td>v1.29.10</td><td>192.168.1.11</td><td>Ubuntu 22.04.5 LTS</td><td>6.8.0-40-generic</td><td>docker://27.3.1</td></tr><tr><td>node1</td><td>Ready</td><td>worker</td><td>3d22h</td><td>v1.29.10</td><td>192.168.1.12</td><td>Ubuntu 22.04.5 LTS</td><td>6.8.0-40-generic</td><td>docker://27.3.1</td></tr><tr><td>node2</td><td>Ready</td><td>worker</td><td>3d22h</td><td>v1.29.10</td><td>192.168.1.13</td><td>Ubuntu 22.04.5 LTS</td><td>6.8.0-40-generic</td><td>docker://27.3.1</td></tr></table> root@master2:~# etcdctl member list --write-out=table <table><tr><th>ID</th><th>STATUS</th><th>NAME</th><th>PEER ADDRS</th></tr><tr><td>5dd5...</td><td>started</td><td>master</td><td>192.168.1.10</td></tr><tr><td>ce406cc1e9a02727</td><td>started</td><td>master2</td><td>192.168.1.11</td></tr></table> 정상적으로 etcd 까지 동기화 되어 있는 모습	NAME	STATUS	ROLES	AGE	VERSION	IP	OS	CONTAINER-ENGINE	CONTAINER-RUNTIME	master	Ready	control-plane	3d22h	v1.29.10	192.168.1.10	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1	master2	Ready	control-plane	43h	v1.29.10	192.168.1.11	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1	node1	Ready	worker	3d22h	v1.29.10	192.168.1.12	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1	node2	Ready	worker	3d22h	v1.29.10	192.168.1.13	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1	ID	STATUS	NAME	PEER ADDRS	5dd5...	started	master	192.168.1.10	ce406cc1e9a02727	started	master2	192.168.1.11
NAME	STATUS	ROLES	AGE	VERSION	IP	OS	CONTAINER-ENGINE	CONTAINER-RUNTIME																																																		
master	Ready	control-plane	3d22h	v1.29.10	192.168.1.10	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1																																																		
master2	Ready	control-plane	43h	v1.29.10	192.168.1.11	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1																																																		
node1	Ready	worker	3d22h	v1.29.10	192.168.1.12	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1																																																		
node2	Ready	worker	3d22h	v1.29.10	192.168.1.13	Ubuntu 22.04.5 LTS	6.8.0-40-generic	docker://27.3.1																																																		
ID	STATUS	NAME	PEER ADDRS																																																							
5dd5...	started	master	192.168.1.10																																																							
ce406cc1e9a02727	started	master2	192.168.1.11																																																							
구성 및 작성 내용 설명 6	실시간 데이터를 시각적으로 표현하며, 대시보드에서 시스템 상태를 쉽게 모니터링																																																									
자료(이미지 외)	 The dashboard displays various system metrics including CPU usage, memory usage, disk I/O, and network traffic. A prominent green gauge shows a value of 41.3%.																																																									